

A Hybrid Graph-Table Interface for Study Planning: Design and an Observational Study of How Students Plan with It

Anonymous Author(s)

Anonymous Affiliation
anonymous@example.org

Keywords: Study Planning, Curriculum Visualisation, Educational Dashboards, Educational Recommender Systems, User Study, Higher Education.

Abstract: Students plan their studies against curricula that encode prerequisites, electives and institutional rules, and the tools that support this planning are split in two: graph-based systems draw the structure of a curriculum but leave the timeline aside, while table-based planners arrange courses across semesters but leave the structure implicit. We design, build and evaluate a hybrid interface in which a browsable curriculum graph and a schedule-managing table write to one shared plan, checked in real time against the regulations of two degree programmes and complemented by a progress dashboard and explainable recommendations. A formative study with six students yields a ranked specification of 13 features carrying 55 requirements. A within-subjects observational study with eleven students, each planning once without the graph and once with it, traces the planning process through a frame-sampled record of the interface, think-aloud transcripts and questionnaires. Students planned in a single checklist-driven loop whose shape did not change with the graph: the graph replaced the course catalogue for finding candidates, courses continued to be committed in the table, and a staging area between the two views became essential. Because the curricula encode few hard dependencies, the graph's value lay in filtered browsing rather than in showing sequences. Self-reported success exceeded audited success, as students deferred to a completion signal that did not model every task constraint. We derive six design implications for interfaces that guide constrained, multi-step decisions.

1 INTRODUCTION

Students in higher education plan their studies against curricula that combine prerequisites, electives and institutional rules. Institutions publish exemplary study plans, but these assume an idealised progression; a single failed examination or a postponed module immediately requires an individual schedule (Judel et al., 2023). Advisors struggle to give evidence-based scheduling advice at scale (Schulte et al., 2017), which matters because the timing and sequencing of courses are associated with time to degree and final grade (Morsy and Karypis, 2019).

Two families of tools address this. Graph-based systems visualise the dependencies a curriculum encodes and leave the timeline aside (Auvinen et al., 2014; Rollande and Grundspenkis, 2013; Trippel and Röpke, 2025); table-based planners arrange courses across semesters and check the result against the regulations, leaving the structure implicit (Hirmer et al., 2022; Judel et al., 2023). The division matters because a student alternates between two different ques-

tions: *which courses could follow from what I have done*, a question about structure, and *does the plan still hold once they are placed*, a question about load, sequence and rules across the whole programme. Where both capabilities have been brought into one tool, dependency information has been embedded in the schedule itself (Röpke et al., 2024; Judel et al., 2023), which ties it to a chronological layout and offers no surface on which the curriculum can be browsed as a structure. To the best of our knowledge, no prior system joins a browsable curriculum graph to a schedule-managing table over one shared plan.

This paper builds and studies such a system: a Graph View draws the curriculum as its containment hierarchy with formal prerequisites as a switchable overlay, a Table View holds the same plan as semester lanes, and both write to one plan that a Compliance Engine checks after every change. Educational dashboards (ED), which report a student's standing (Schwendimann et al., 2017; Verbert et al., 2020), and educational recommender systems (ERS), which suggest what to take next (Bhumichitr et al., 2017; Es-

teban et al., 2020), are established but almost always built separately (Bodily and Verbert, 2017); whether they belong inside a student's own planning workflow is a question we put to students rather than settle in advance. Three research questions guide the work:

- **RQ1** What are students' needs and resulting requirements for a hybrid graph-table interface that supports programme-level study planning?
- **RQ2** How can such an interface be designed and implemented from those requirements?
- **RQ3** How do students plan with the hybrid interface, and what does the graph-based view contribute to their planning?

The paper contributes (i) a requirements specification of 13 features and 55 requirements derived from a formative study and traceable to interview evidence; (ii) the design and implementation of a web-based tool that unites a curriculum graph and a schedule-managing table over a single plan, checks it against the regulations of two degree programmes in real time, and couples a progress dashboard with explainable recommendations inside the planning workflow; and (iii) a within-subjects observational study of eleven students, observed at the level of the interface itself. Its central finding is that the graph enters an established tabular workflow at exactly one point, the search for candidate courses, and that the staging area between the two views, rather than either view, is what makes a shared-plan hybrid work.

2 RELATED WORK

Hirmer et al. (2022) frame study planning as selection and temporal coordination, distinguishing short-term planning, avoiding timetable overlaps within a semester, from long-term planning, finding reasonable module sequences across semesters. This work addresses long-term planning only, at programme-level granularity, at the intersection of curriculum visualisation, ED and ERS.

2.1 Graph-Based Curriculum Visualisation

A substantial body of work models a curriculum as a graph to make its dependencies visible, from *ViCurriAS*, which shows course dependencies alongside student progress in advising sessions (Zucker, 2009), to graph-based frameworks for personalised study planning (Rollande and Grundspenkis, 2013). *STOPS* (Auvinen et al., 2014) models the curriculum as a

graph of learning outcomes rather than courses and serves both students building plans and staff maintaining the model. Trippel and Röpke (2025) present an elective-focused graph distinguishing required and recommended edges; its requirements were elicited from institutional stakeholders, and the prototype was well received by students. Across this paradigm, scheduling support is thin: *STOPS* lets a student drag each course onto a teaching period and tracks credit totals, and *ViCurriAS* records a semester per course, but none manage workload across a plan.

2.2 Table-Based Planning Tools

A complementary body of work prioritises scheduling using tabular representations. *CourseRank* (Bercovitz et al., 2009) combines official data with user content and offers a planner, a requirement tracker and a peer-based recommender. Hirmer et al. (2022) elicit requirements from twelve computer-science students; their prototype comes closest to the hybrid proposed here, a semester-by-semester schedule from which a click on a module opens a detail view drawing that module's dependencies, but the graph is confined to one module at a time. *AISudyBuddy* (Röpke et al., 2024) and the prototype it absorbed (Judel et al., 2023) integrate dependency awareness directly into a semester-by-area matrix, highlighting prerequisites and warning on rule violations inside the table. Tying dependencies to a matrix aligns them with a chronological layout; a dedicated graph view offers the complementary perspective of exploring the structure a curriculum encodes without the constraints of a timeline.

2.3 Dashboards and Recommenders

Course Signals (Arnold and Pistilli, 2012) reported improved retention from a traffic-light risk indicator, a result Caulfield (2013) questions because the analysis did not control for the number of courses a student took, and Chaturapruet et al. (2018) found that showing students grade distributions lowered GPA in a randomised encouragement design. Neither result argues against reporting a student's standing; both concern what a student does when the report is treated as authoritative, a property we examine in our own compliance layer.

ERS research is largely organised by technique, from content-based and hybrid filtering (Esteban et al., 2020) to collaborative filtering (Bhumichitr et al., 2017). A smaller strand emphasises explainability: *CourseQ* (Ma et al., 2021) showed in a within-subjects study that a visual, interactive explanation

improved perceived accuracy, transparency and trust with the algorithm held constant. Reviews converge on the difficulty of making dashboard data actionable rather than merely visible (Verbert et al., 2020), and Teasley (2017) notes that the moderators of feedback effects are poorly understood. Bodily and Verbert (2017) locate the gap structurally: of 93 articles on student-facing systems, only 17 % described systems integrating both feedback on past performance and actionable next steps, and only 6 % reported a needs assessment.

2.4 Positioning

Two gaps follow. Graph-based tools make dependencies visible but do not manage workload across a plan; table-based tools manage schedules and surface dependencies only locally. And student-facing ED and ERS are each established, but rarely integrated and seldom grounded in needs assessments. This work addresses the first gap with a hybrid interface over one synchronised plan, grounded in requirements elicited from students, and the second by putting the ED–ERS combination to those same students and building what they asked for. The design is guided throughout by the nine design principles for study-planning assistants of Bartel et al. (2024).

3 FORMATIVE STUDY

To ground the design in students’ planning practices (RQ1), we conducted a formative study around an interactive but non-functional prototype. The prototype represented the hybrid concept as three linked spaces: a Table View laying the plan out as semester lanes fed from a searchable catalogue, with running credit totals; a Graph View showing the same curriculum as a node-link structure of exam subjects, modules and courses with display filters; and recommendation actions reachable from both. It had no backend, no persistent plan and no rule checking. The hybrid concept itself was a starting assumption motivated by the literature, so the study elicited requirements *within* that concept rather than testing it against alternatives.

Participants and Procedure. Six students of the Faculty of Informatics of a European technical university took part, three from the Bachelor’s programme in Computer Science and three from the Master’s programme in Software Engineering, chosen to span different degrees of curriculum familiarity. Each session combined a semi-structured interview with a think-aloud walkthrough of the prototype in four phases:

consent, current planning behaviour, prototype exploration, and reflection. Sessions were audio-recorded and transcribed verbatim.

Analysis. The analysis followed the *codebook* branch of thematic analysis (Braun and Clarke, 2022), chosen because its output had to remain traceable to the excerpts behind it in order to carry forward into numbered requirements; the codebook followed the charting procedure of framework analysis (Gale et al., 2013). Excerpts were assigned inductively developed initial codes, each stored with a verbatim quote and interview ID (435 codes across six interviews), and consolidated into 64 themes, each a design-relevant need category. Themes were reviewed against five scope boundaries fixed in advance (no personalised automated planning; programme-level granularity; no live link to institutional systems; no community features; recommendation quality not a contribution): 51 were carried forward, eleven ruled out of scope, two set aside as procedural. Prevalence across participants was recorded only to sequence implementation, not as evidence of importance (Braun and Clarke, 2022).

Findings and Requirements. Five themes recurred in all six interviews: course choice weighed against feasibility; unification of the institutional and personal tools students otherwise consult separately; automatic checking of curriculum rules instead of manual tracking; recommendations that are grounded and personally relevant instead of generic; and the representation of prerequisites and their order. Each theme was read for what the system must support, yielding requirements grouped into features, with the chain *evidence* → *code* → *theme* → *requirement* → *feature* kept traceable in both directions. The result is 13 features carrying 55 requirements (Table 1). The first four form the planning core that defines the hybrid concept; the next four add a judgement layer around it; the last five concern how the representation behaves once populated.

4 THE HYBRID INTERFACE

This section answers RQ2. Every design decision is grounded in a requirement, a tolerated prototype assumption, published guidance, notably Bartel et al. (2024) and Nielsen (1994), or a stated design judgement, at the encoding and interaction level of Munzner’s nested model (Munzner, 2009). Figures 1 and 2 show the two views.

Table 1: The 13 features in descending order of importance score (summed prevalence of their themes), with the number of requirements each carries.

ID	Feature	Score	Reqs.
F01	Unified data and tool integration	20	5
F02	Multi-level roadmap and progress dashboard	19	6
F03	Curriculum rules and compliance engine	18	5
F04	Graph-to-table planning workflow	15	6
F05	Feasibility planning	14	3
F07	Evidence-based, explainable recommendation	13	6
F06	Onboarding and guided planning transition	12	2
F08	Constraint-aware replanning	9	4
F09	Overwhelm-safe browsing	6	3
F10	Graph interpretability	5	4
F11	Motivating visual UI, low emotional load	5	4
F12	Recommendation presentation UX	4	3
F13	Spatial organisation and grouping	3	4

4.1 Shell and Plan Model

Participants described planning as working at a desk, “you just push things around”, which presupposes simultaneous access to several activities. The interface is therefore divided into three fixed, simultaneously visible panels: a left panel holding the Course Catalogue in Table View, the filter controls in Graph View, and an independently toggled Recommendation Panel; a centre canvas hosting the active view; and a right panel holding the Dashboard, which reports compliance and progress regardless of the active view. A fixed region per function follows the recognition-rather-than-recall heuristic (Nielsen, 1994); a modal Dashboard was rejected because compliance feedback surfaced only on request would let violations accumulate silently.

Only one of Table View and Graph View occupies the centre at a time, because both need the full width and because participants differentiated the two by purpose, turning to the graph to “find out what is there” rather than to plan. Both are rendered on the same pannable, zoomable canvas with identical course cards. Underneath, a single plan model is the only authority: both views are projections of it, and the Compliance Engine, the Dashboard and the recommender all read from it. Every course is in exactly one of four lifecycle states, *not planned*, *parked*, *planned* or *done*, and every action is offered both as drag-and-drop and as a context-sensitive button showing only the transitions valid in the current state.

The curriculum is imported as a strict four-level

hierarchy, programme → exam subject → module → course, with official course codes, following the centralisation and university-identifier principles of Bartel et al. (2024). The two supported programmes define obligation differently (the Bachelor’s has quota-based narrow and broad electives, the Master’s has core modules that gate electives), so the data model encodes each programme’s semantics separately and exposes them uniformly as *mandatory* / *constrained elective* / *fully elective*, drawn as three border weights so that hue stays free for a subject-colour hierarchy: full saturation on the exam-subject node, a 20 % tint on the module, a coloured border on the course card. Planning status uses fill lightness and border saturation; red is reserved for violations and the destructive *remove* action.

4.2 Graph View

The Graph View renders the curriculum tree left to right across the four depth levels, with exam subjects stacked as horizontal bands, so that horizontal position encodes depth and vertical band membership encodes the exam subject. Participants had asked for a left-to-right layout “because we read left to right” and whether vertical position meant anything; the prototype’s radial layout was replaced for both reasons, and a force-directed layout rejected because recomputed positions would destroy spatial memory. Students may drag nodes horizontally within their column and the offset persists; the vertical axis is system-managed. The curriculum’s hard prerequisites are drawn as a switchable overlay on the same layout.

A fully expanded Master’s programme contains well over 100 nodes, which “kind of explodes”. Every exam subject and module is individually collapsible, the default is collapsed to the exam-subject level, and a collapsed module is replaced by a placeholder of identical dimensions so nothing reflows. Filtering is a purely visual, reversible layer over six dimensions: obligation type, credit range, course type, exam subject, planning status and term availability; values within a dimension combine disjunctively, dimensions conjunctively.

4.3 Table View and Parking Stage

The Table View lays the plan out as one lane per semester, each header showing the semester label and a continuously updated credit total. A course is assigned by dragging it into a lane, or through a Semester Picker anchored to the card that lists only the semesters in which the course is offered, warns where the student’s own workload target would be ex-



Figure 1: Table View. Semester lanes with running credit totals hold the plan; the Parking Stage on the left holds courses that have been shortlisted but not yet assigned to a semester. Card border colour encodes the exam subject, border weight the obligation type, and the footer carries credits, obligation and status.

ceeded, and always offers one further future semester so the timeline extends without a configuration step.

Students separate exploring options from committing them to a timeline: finding interesting courses is “cool as a first step, not thinking yet about which semester”. The tool is therefore built around a three-stage workflow, *browse* (graph or catalogue), *shortlist* (Parking Stage), *assign* (table), with the middle stage optional. The Parking Stage sits directly left of the semester lanes on the same canvas (Figure 1), omits chronological and workload indicators, and heads the list in every Semester Picker so that a course found anywhere can be held without a semester decision. A sidebar list was rejected because *parked* is an intermediate state between *not planned* and *planned*, and its position should reflect that progression.

4.4 Compliance Engine and Dashboard

Manual rule checking was described as effortful and unreliable (“Excel doesn’t know your requirements”). A single compliance check runs after every planning action and reports the plan’s status, its hard violations and its soft warnings. Hard violations (a broken hard prerequisite, a semester above the credit ceiling) revert the action and appear as persistent notices; soft warnings (an under-loaded semester, a sub-optimal elective sequence) are dismissible advisories, following the requirements-versus-recommendations distinction of Judel et al. (2023). Messages name the course, semester and action that triggered them.

Regulations are encoded as data, one rule set per programme.

The Dashboard aggregates progress at the four granularities students named, programme, category, semester and module, in collapsible and reorderable sections, with two modes separating what is *planned* from what is *done*. Its programme level carries the checklist of missing requirements. Because every action recomputes it, dropping a course on the canvas already simulates “what happens if I add this”. Per-semester load is checked against two values held in a personal profile, a recommended target and a maximum, defaulting to 30 and 42 credits; the target is advice, the maximum a limit. The tool never re-balances a plan on the student’s behalf.

4.5 Recommendations

Participants were sceptical of recommendations “based on nothing”, so the design follows the finding of Ma et al. (2021) that inspectable explanations raise trust with the algorithm held constant. Rather than a blended score, recommendations come from six named *families*, each of which states its reason on the card in the student’s terms: interest match (“matches your interest in machine learning”), curated content similarity, sequence or dependency, internship relevance to a stated career direction, follow-on from a completed course, and peer choice, the last two computed over a cohort of prior plans. Where a signal is weak, the card says so. Recommendations are con-

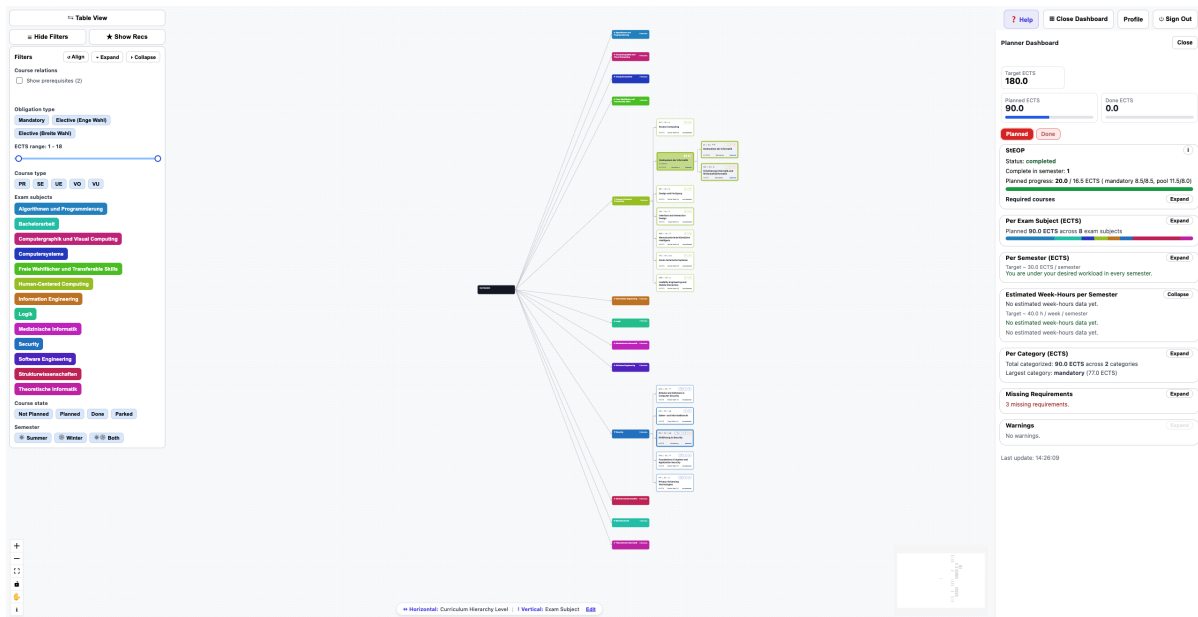


Figure 2: The three-panel shell with the Graph View active: filter controls on the left, the curriculum tree in the centre drawn left to right from programme to exam subject to module to course with two subjects expanded, and the Dashboard on the right reporting credits, per-subject and per-semester progress, missing requirements and warnings.

fined to an independent left-hand panel adjacent to the catalogue; nothing is added to the canvas, and accepting a card uses the same two paths as any course. Recommendation quality is explicitly not a contribution: the behavioural families run on a synthetic cohort, and the engine exists to exercise the workflow of explainable suggestions.

4.6 Implementation

The tool is a single-page web client (React, both views on one React Flow canvas) backed by a stateless Python service (FastAPI) and a PostgreSQL database. The recommender scores candidates through six channels behind one protocol, passes the top 100 to the Compliance Engine for trial placement, discards any that would introduce a new error or warning, and returns at most 15. Every state change is timestamped. Six defects observed in the study were closed afterwards, with regression tests; none of those changes was in front of a participant.

5 EVALUATION STUDY

The study (RQ3) asks how a curriculum graph alters the planning process of students equipped with an established tabular workflow; neither the premise that the graph is useful nor the intended division of labour had been tested. We ran a within-subjects ob-

servational study ($N = 11$) in which each participant completed comparable planning tasks first with the tabular baseline alone, then with the graph enabled. The design prioritises observational fidelity over outcome comparison: frame-level logging paired with think-aloud transcripts lets the graph’s contribution be traced within the process. The fixed order is deliberate: the study asks how a graph enters a workflow a student has already formed, so the baseline had to be formed first, and counterbalancing would not remove the confound, since where a treatment is differently effective at different levels of practice it leaves treatment effects mixed with treatment-practice interactions (Greenwald, 1976). Because graph availability and second-task exposure are collinear, we attribute no change in speed or outcome to the graph and ask only whether it displaces the established workflow.

5.1 Participants and Procedure

Twelve sessions were run and eleven analysed; one participant declined screen recording. Participants were students of the same faculty as the formative study, enrolled in one of the two target programmes, from their second to eighth semester; four reported regular prior experience with study-planning tools, one little, six none. Each completed two persona-framed planning tasks in a fixed order: *Scenario A*, a Cybersecurity persona, with the graph unavailable (everything else fully available), and *Scenario B*, an

Artificial Intelligence persona, with the full tool. Each brief stated only the persona’s constraints, a per-semester credit band and one reduced-load internship semester, plus guidance on preferred module types; the curriculum’s own hard requirements were deliberately not stated and were encountered only inside the tool, so the Dashboard was the sole channel through which curriculum constraints reached participants. Sessions were screen- and audio-recorded in a fixed sequence (demographics, consent, video tutorial, Scenario A, post-task questionnaire, Scenario B, post-task questionnaire, open exploration, final questionnaire, semi-structured interview), with think-aloud throughout and a moderator present to unblock operational questions. Post-task questionnaires held three items on task success, plan satisfaction and difficulty plus six domain-adapted items from the Technology Acceptance Model (Davis, 1989); the final questionnaire was the 26-item User Experience Questionnaire (UEQ) (Laugwitz et al., 2008).

5.2 Analysis

Two records were collected per participant and triangulated. Screen recordings were sampled every ten seconds, yielding 2,179 frames across the 22 runs, and each frame was coded against a fixed scheme capturing the active surface, open panels, sidebar contents, per-semester credits, Dashboard state and transient banners. Frame coding was executed by a locally hosted vision-language model under a strict protocol (one independent row per frame, no interpolation); its reliability was checked against the first author’s manual coding of three complete runs (183 frames). Fixed-interval sampling was chosen because the quantities reported are durations. The method does not see events shorter than ten seconds and records state rather than intent, so every visual code was read against the transcript. Transcripts and recordings were analysed with a deductive codebook classifying usability problems (E-P), positive findings (E-G) and neutral observations (E-N); each code maps to an ISO 9241-11 dimension (International Organization for Standardization, 2018), and problems carry a severity on Nielsen’s five-point scale (Nielsen, 1993). Frequencies are reported as n/m , where m is the number of participants for whom the code could be decided.

From the frame labels we derived a state model of the planning process: a state is the primary surface together with the two panels that can vary beside it. In the Table View the Dashboard and the sidebar can each be open or closed, giving DASH+CAT (Dashboard open, catalogue in the sidebar: the checklist-

driven working state), CAT (catalogue only: browsing), DASH (Dashboard only: reading status) and TABLE (neither). In the Graph View the sidebar is always the filter panel, so GRAPH is a single state; OFF covers the study guide and questionnaires. The Recommendation Panel stands open across any surface without changing where the student is working, so it is counted as an overlay rather than a state.

6 FINDINGS

6.1 The Planning Loop

Planning under this tool is a single checklist-driven loop with the same shape for every participant: read what the Dashboard says is still missing, find a candidate that closes the gap, place it, and return to the list. The missing-requirements checklist drove task completion for all eleven participants (E-G15, 11/11), and all seeded their plans with the pre-filled mandatory backbone (E-G01, 11/11). P09 opened Scenario A by reading the count aloud, “22 missing requirements, wow, that is a lot”, and resolved them item by item. For the most junior participant the loop only emerged after the moderator pointed to the tracker (“exactly that, here I see what I need”).

The loop alternates between the checklist-reading state DASH+CAT and a candidate-finding state, which in Scenario A is the catalogue (Table 2). We name its three stages *drive* (reading the checklist), *find* (turning a gap into candidates) and *commit* (placing a candidate). The transition CAT→DASH+CAT accounts for 27.5 % of all state changes in Scenario A and the reverse for 17.4 %, so this single pair makes up 44.9 % of all logged transitions. The cycle is tight: the median path from an open Dashboard back to the Dashboard is 2.0 transitions ($n = 38$ returns), the shortest possible round trip.

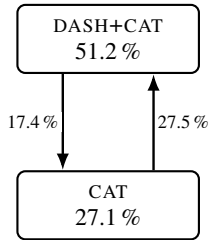
6.2 What the Graph Changes

The graph alters the loop at exactly one point: it replaces the catalogue as the surface on which a missing requirement becomes candidate courses, and leaves everything downstream unchanged (Figure 3). In Scenario B the cycle alternates between the Dashboard and the graph; DASH+CAT→GRAPH and its reverse each account for 19.3 % of state changes (38.6 % combined), against the 44.9 % the catalogue-Dashboard cycle held in Scenario A. Occupancy shows the same substitution: catalogue-only occupancy drops from 27.1 % to 6.7 % while graph occupancy rises to 26.6 %, close to the share the cat-

Table 2: State occupancy, entries and mean dwell per entry over the eleven runs of each condition (1,197 frames in Scenario A, 982 in Scenario B). The catalogue and the graph exchange places. *The Recommendation Panel is an overlay counted independently of the states.

State	Scenario A			Scenario B		
	Occ. %	Entr.	Dwell s	Occ. %	Entr.	Dwell s
DASH+CAT	51.2	38	161	46.5	51	90
CAT	27.1	40	81	6.7	22	30
GRAPH	—	—	—	26.6	42	62
DASH	5.5	11	60	8.2	8	101
TABLE	0.6	5	14	2.0	3	67
OFF	15.6	26	72	9.9	35	28
RECS*	24	23	125	20	8	244

Scenario A (graph off)



Scenario B (graph on)

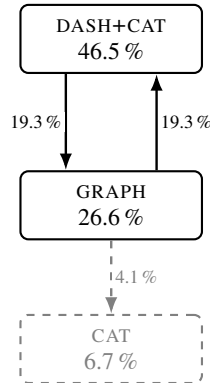


Figure 3: The dominant planning cycle in each condition, with each transition’s share of all state changes and each state’s occupancy. Shape and period are unchanged; only the candidate-finding surface differs.

atalogue vacates; the catalogue was entered 40 times across the Scenario A runs and the graph 42 times in Scenario B, at comparable dwell. The catalogue survives only as a side branch out of the graph. At the participant level, catalogue occupancy falls in ten of eleven cases, from a median of 27.4 % to 4.5 %. The cycle keeps its shape and period: the median return path to the Dashboard remains two transitions in both conditions. Two things separate this from a practice effect: the graph was made available rather than prescribed, and the catalogue remained available and unchanged. Growing familiarity accounts for a faster second run, not for the abandonment of a surface used competently in the first.

Query Support Without Commit Support. The graph supports the *find* step but structurally inhibits the *commit* step. P02 read a missing requirement from the Dashboard, switched to the graph, applied the narrow-elective filter, harvested candidates into

the Parking Stage, and returned to the table to place them. Others combined filters for queries the catalogue cannot express, such as term availability with “not planned”. Conversely, commitment forced a return to the table because the graph renders no semester dimension, so placing a course from a graph node relies on working memory. P10: “adding directly from here I would probably not do, because I do not know which semester has room”. P06 blindly pushed a semester past its credit cap from the graph, a breach discovered only on returning to the table.

What Does Not Change, and What Cannot Be Attributed. Placement accuracy remained stable: the share of placement attempts ending in a system rejection moved from a median of 16.7 % in Scenario A to 13.6 % in Scenario B, decreasing for seven participants and increasing for four, a split consistent with chance (sign test, $p = 0.55$). The graph changed *where* candidates were found and left *how* they were committed untouched. The remaining differences are documented but left unattributed because of the fixed order: placements per ten minutes rose from a median of 7.8 to 13.9 (in ten of eleven participants) and sampled time in the planner fell from 20.8 to 13.3 minutes despite a longer second task; six of eleven Scenario A runs and seven of eleven Scenario B runs met every constraint of their brief.

6.3 Around the Loop: Staging, Recommending, Configuring

The Parking Stage. The Parking Stage is what lets two surfaces work on one plan without either doing the other’s work, and every participant used it to shortlist before committing (E-G06, 11/11). P04 announced the strategy, “I will put everything into the parking zone and then go back to the table and assign each to the right semester”, and the recordings show it executed: P09 moved nine courses from graph nodes into the stage in a single episode. That the stage sits outside the plan’s accounting is both its function and a source of error: parked credits are excluded from the planned total and nothing on screen says so (E-P38, 10/11). P02 planned against 102 credits while 24 sat parked; two others ended a run with credits parked while the Dashboard reported a complete plan. Nine participants asked for the stage to be visible from inside the graph, where it is not (E-P37, 9/11).

The Recommendation Panel. The panel was one of the most occupied surfaces, open for 24 % of frames in Scenario A and 20 % in Scenario B, yet

it never entered the cycle. Active stations show frequent entries and short dwells; the panel shows the opposite, 23 entries at a 125 s mean dwell in Scenario A and eight at 244 s in Scenario B, with 71 % of its open time in nine episodes over three minutes. Use divided by how far a student could judge the curriculum alone: ten participants incorporated recommendations (E-G02, 10/11), but seven ultimately bypassed the panel for the checklist and catalogue search (E-N05, 7/11), six of them after having used it successfully earlier, a deliberate choice rather than a discovery failure, while the two participants lacking the knowledge to judge modules independently leaned on the panel and the Dashboard as compensation (E-N06, 2/11). Three defects contextualise this: the overlay occluded the lanes it was meant to populate (E-P11, 6/11); the internships family rendered a bare empty state when the profile's career field was blank (E-P42, 4/11); and four of nine participants could not tell which profile input had triggered a suggestion, although every card gave a reason (E-P68, 4/9).

User-Configured Thresholds. Per-semester load is the one rule checked against profile values rather than curriculum data. Aligning them to the brief was part of the task, but a profile left untouched is checked against the defaults with nothing on screen to say so. P01 changed them within four minutes and discovered the asymmetry in the same breath ("I will set it to 33 straight away, more is not allowed, and is there a minimum? No"); P06 completed Scenario B with a semester at 30 credits against a stated band of 21 to 27 while the Dashboard reported no warning, because the recommended value was still 30 (E-P32, E-P57, 11/11). Students trusted the verdict in both cases.

6.4 What Worked and Where the Loop Breaks

Five positive findings are universal: the pre-filled plan removes the blank page, the checklist drives completion, the Parking Stage holds candidates, real-time compliance feedback is noticed and used (E-G04, 11/11), and the graph's filters are valued once found (E-G07, 11/11); nine of eleven used the graph for interest-driven browsing (E-G10) and six said it clarified module composition (E-G17). The most costly problems cluster at the three stages of the loop. At the *drive* stage, status was not glanceable: requirement and credit-gap status lived behind collapsible panels (E-P10, 11/11) and was stated in prose rather than as a figure (E-P03, 7/11), and across the four most recent sessions the warnings panel was never opened

while its counter climbed to between two and five. At the *find* stage the catalogue could not be narrowed, since filtering by category, obligation or credits existed only in the graph's sidebar (E-P04, 11/11), so students in Scenario A fell back on free-text search. At the *commit* stage the lanes did not show whether a semester was winter or summer, so students guessed the term and rejections and rework followed (E-P29, 11/11), the most frequently coded friction: a visualisation deficit rather than a validation error, since the rule was evaluated correctly but the information needed to avoid the error was withheld at the point of decision. Further high-severity problems were the workload band not being enforced (E-P26), completion being signalled over an incomplete constraint set (E-P34), the graph carrying no semester dimension (E-P47, 10/10), and unclear curriculum terminology (E-P01, 4/11).

6.5 Questionnaires

The UEQ describes a tool judged capable before engaging: pragmatic quality +2.28 outscores hedonic quality +2.00 on the -3 to +3 scale, attractiveness (+2.61) and efficiency (+2.59) lead, and the two weakest scales map onto the qualitative record. Perspicuity (+1.82, SD 0.70) reflects the learnability and terminology friction above; novelty (+1.77, SD 0.78) reflects that participants read the tool as a better version of what they already did (P04: "until now I only edited Excel files manually, I find this really great"). The adapted TAM items rate high in both conditions (ease of use 5.59 and 6.09 of 7; usefulness items 6.09–6.64), read descriptively given the fixed order. The instructive result is a single item: satisfaction with the finished plan is the only rating that falls, from 6.27 to 5.91, and the two largest drops belong to the participants whose Scenario B plans went furthest wrong, whereas on the same sheet the success item rated seven of the eight failed runs successful. Students' judgement of the plan tracked its measured quality; their judgement of their own success did not.

6.6 The Plans Produced, Against Self-Report

Every finished plan was audited against its brief. Of 21 resolved runs, 13 met every constraint. Of the eight failures, four missed the reduced-load internship semester, two fell short of the degree total, and two exceeded the maximum workload through the profile mismatch above. Participants rated seven of the eight failed runs as successful, so self-reported success tracked *felt* rather than actual completion (E-

N03, 9/11). In all 22 runs the Dashboard ultimately reported zero missing requirements, correctly against the curriculum's rules and silently against the task's. P02 correctly reserved an empty lane for the internship; because the system has no way to register an internship semester, the Dashboard flagged a missing credit total, and P02, trusting the interface over the brief, filled the lane to clear the warning. An unrepresented constraint is not merely left unverified; it actively competes with the constraints the checklist does display, and the interface won in every case observed.

7 DISCUSSION

The design rested on several untested assumptions; the evaluation speaks to each. The result is a *process*, not a *feature*: one loop in which the pre-fill seeds a plan, the checklist drives it, catalogue, graph and recommendations propose candidates, the Parking Stage holds them and the table commits them, externalising exactly the part of programme planning students cannot hold in their heads. *The Dashboard as compass* did more than intended: because the briefs stated no curriculum rules, participants met the curriculum only through it, so it did not support an understanding of the curriculum, it constituted one, and what it did not hold them to did not exist for them. *A staging area between finding and committing* mattered more than the design claimed: a shared-plan hybrid does not need each view to do everything, but it needs an explicit place for a decision made and not yet committed. *Graph for exploration, table for commitment* held with a narrower scope and for a different reason: graph activity was directed querying, the division was driven by missing context (no lanes, no totals) rather than by representational limits, and the query step's value came largely from the filter panel, which leaves open how much would survive if the table shared those filters. *Drawing only what the curriculum fixes* was sound as a rule and empty in application: the Bachelor's programme fixes no ordering between two named courses, so the overlay was empty rather than sparse, and four participants asked for ordering at the point of placement. *Guide rather than block* was supported, but the tiering was asymmetrical, a hard block above the ceiling and a dismissible warning below the floor, assigned by how a breach sounds rather than what it costs, and the thresholds came from a profile whose defaults silently validated plans against the wrong values. *Recommendations with reasons* held for use and half held for reasons: they served the least expert users most, while confident users wanted a different route to the same end.

7.1 Design Implications

Six rules follow for interfaces that guide a user through a constrained, multi-step decision, of which a study planner is one instance.

- **An authoritative signal must cover every actual constraint.** Users do not read a partial completion signal as partial. Pending work should be counted or visibly excluded, and active defaults announced.
- **Tier rules by the cost of the breach,** not by how it sounds; opposing bounds of one constraint should carry the same friction.
- **Allow commitment only from views that show the consequences.** A view that cannot show what an action alters should be restricted to finding and staging.
- **Put decision-making data where the decision happens.** Filters lived in the graph while placement happened in the table; status lived behind collapsed panels.
- **Show valid targets before an action is attempted,** rather than rejecting invalid attempts afterwards.
- **Treat guided and self-directed workflows as equals,** and make automated guidance explain its inputs so confident users can audit rather than discard it.

7.2 Relation to Prior Work

The findings are *consistent with* the friction points Bartel et al. (2024) report for study-planning assistants: reading institutional terminology and finding nested filters. They *qualify* the premise of the graph-visualisation literature (Zucker, 2009; Auvinen et al., 2014; Trippel and Röpke, 2025) that a curriculum carries a prerequisite network worth drawing: the curricula studied here carry little of one, the graph draws containment rather than sequence, and a prerequisite advisory arose once in 22 runs. They *extend* the AIS-tudyBuddy lineage (Röpke et al., 2024; Judel et al., 2023; Trippel and Röpke, 2025) by observing the process of forming a plan rather than its reception or its post-hoc paths: positive reception is reproduced, but it does not predict the graph's role, which was a query surface that displaced the catalogue.

The completion finding gives the dashboard critique a counterpart at the level of the individual student. Caulfield (2013) objected that Course Signals was credited beyond what had been checked; here the Dashboard reported no missing requirements, correctly for the curriculum and silently for the task, and

participants read that report as completion in preference to their own model. Parasuraman and Riley (1997) describe the hazard as misuse of automation, over-reliance governed among other things by the saliency of a system's state indicators; a completion checklist is exactly such an automation, continuous, salient and silent about what it has not checked. The split in recommendation use speaks to the integration called for by Bodily and Verbert (2017): the components were available together but not used as a whole, with a division of labour dictated by curriculum confidence, the same line Ma et al. (2021) found between students with and without clear goals. Finally, the displacement of the catalogue by the graph is an instance of cognitive fit (Vessey, 1991): spatial tasks are served by graphs, symbolic tasks by tables, and the hybrid makes that distinction physical.

7.3 Limitations

The requirements come from six students at one faculty, elicited around a prototype that already assumed a graph-and-table interface, so the formative study shaped how such an interface should behave without testing whether students want one. The fixed task order entangles the graph's introduction with practice effects, which is why only displacement and rejection share are interpreted. Eleven computing students at one institution, likely more comfortable with node-link diagrams than the general student body, bound generalisation, and any pattern by expertise rests on very small subsets. The first author designed the tool, moderated the sessions and coded the results, so the near-ceiling questionnaire scores are vulnerable to demand characteristics, which is why the screen recordings are treated as the stronger evidence; participants planned for a fictional persona under a time limit. Frame sampling at ten seconds misses shorter events and intent, and frames were coded by a vision-language model checked against manual coding of three runs.

8 CONCLUSION

We designed, built and evaluated a hybrid graph-table interface for programme-level study planning and observed how students use it. RQ1 is answered by a specification of 13 features and 55 requirements traceable to interview evidence. RQ2 is answered by the artefact, whose central decision, a single authoritative plan model from which both views and all feedback derive, is what makes the views projections of one plan rather than two plans to reconcile. RQ3 is an-

swered by a consistent checklist-driven loop in which the graph substitutes for the catalogue at the query step, largely through its filters, and does not support the commit step; by a staging area that proved essential as the hinge between the views; by recommendations that were used heavily at first and set aside once students could judge the curriculum themselves; and by an over-reliance on the tool's completion signal that let constraints the checklist did not model fall out of working memory.

The findings generalise in three widths: the completion finding does not depend on study planning at all; the loop, the pre-filled backbone and the split of recommendation use by confidence need only a checklist and a recommender; and the graph at the query step, its dependence on its filters and the Parking Stage as the hinge are specific to hybrids. Each is a hypothesis for a larger study, and one is directly falsifiable: if the graph's value lies in its filter panel, adding those filters to the table should render the graph largely optional. Because the system timestamps every state change, the process account can be scaled into a log study of students planning their actual degrees. Where curricula encode few hard dependencies, future work should reconsider what a curriculum graph ought to display, perhaps broad eligibility gates rather than a sparse prerequisite network, and should test expertise-moderated tool use with a sample drawn from beyond computing programmes.

REFERENCES

- Arnold, K. E. and Pistilli, M. D. (2012). Course Signals at Purdue: Using Learning Analytics to Increase Student Success. In *Proceedings of the 2nd International Conference on Learning Analytics and Knowledge (LAK '12)*, pages 267–270. Association for Computing Machinery.
- Auvinen, T., Paavola, J., and Hartikainen, J. (2014). STOPS: a graph-based study planning and curriculum development tool. In *Koli Calling '14: 14th Koli Calling International Conference on Computing Education Research*, pages 25–34. Association for Computing Machinery.
- Bartel, L., Ochs, M., Hirmer, T., and Henrich, A. (2024). Design Principles for a Study Planning Assistant in Higher Education. In *Conference on Human Information Interaction and Retrieval*, pages 243–253. Association for Computing Machinery.
- Bercovitz, B., Kaliszan, F., Koutrika, G., Liou, H., Mohammadi Zadeh, Z., and Garcia-Molina, H. (2009). CourseRank: a social system for course planning. In *2009 ACM SIGMOD International Conference on Management of data*, pages 1107–1110. Association for Computing Machinery.

- Bhumichitr, K., Channarukul, S., Saejiem, N., Jiamthapthaksin, R., and Nongpong, K. (2017). Recommender Systems for university elective course recommendation. In *14th International Joint Conference on Computer Science and Software Engineering (JCSSE)*, pages 1–5. IEEE.
- Bodily, R. and Verbert, K. (2017). Review of Research on Student-Facing Learning Analytics Dashboards and Educational Recommender Systems. *IEEE Transactions on Learning Technologies*, 10(4):405–418.
- Braun, V. and Clarke, V. (2022). *Thematic Analysis: A Practical Guide*. SAGE Publications.
- Caulfield, M. (2013). A simple, less mathematical way to understand the course signals issue. <https://hapgood.us/2013/09/26/a-simple-less-mathematical-way-to-understand-the-course-signals-issue/>.
- Chaturapruek, S., Dee, T. S., Johari, R., Kizilcec, R. F., and Stevens, M. L. (2018). How a data-driven course planning tool affects college students' GPA: evidence from two field experiments. In *L@S '18: Fifth (2018) ACM Conference on Learning @ Scale*, pages 1–10. Association for Computing Machinery.
- Davis, F. D. (1989). Perceived usefulness, perceived ease of use, and user acceptance of information technology. *MIS Quarterly*, 13(3):319–340.
- Esteban, A., Zafra, A., and Romero, C. (2020). Helping university students to choose elective courses by using a hybrid multi-criteria recommendation system with genetic optimization. *Knowledge-Based Systems*, 194:105385.
- Gale, N. K., Heath, G., Cameron, E., Rashid, S., and Redwood, S. (2013). Using the framework method for the analysis of qualitative data in multi-disciplinary health research. *BMC Medical Research Methodology*, 13(1):117.
- Greenwald, A. G. (1976). Within-subjects designs: To use or not to use? *Psychological Bulletin*, 83(2):314–320.
- Hirmer, T., Etschmann, J., and Henrich, A. (2022). Requirements and Prototypical Implementation of a Study Planning Assistant in CS Programs. In *International Symposium on Educational Technology (ISET)*, pages 281–285. IEEE.
- International Organization for Standardization (2018). ISO 9241-11:2018 — ergonomics of human-system interaction — part 11: Usability: Definitions and concepts.
- Judel, S., Röpke, R., Azendorf, M., and Schroeder, U. (2023). Supporting Individualized Study Paths Using an Interactive Study Planning Tool. In *21. Fachtagung Bildungstechnologien (DELFI)*.
- Laugwitz, B., Held, T., and Schrepp, M. (2008). Construction and evaluation of a user experience questionnaire. In *HCI and Usability for Education and Work (USAB 2008)*, volume 5298 of *Lecture Notes in Computer Science*, pages 63–76. Springer.
- Ma, B., Lu, M., Taniguchi, Y., and Konomi, S. (2021). CourseQ: the impact of visual and interactive course recommendation in university environments. *Research and Practice in Technology Enhanced Learning*, 16(1):18.
- Morsy, S. and Karypis, G. (2019). A Study on Curriculum Planning and Its Relationship with Graduation GPA and Time To Degree. In *The 9th International Learning Analytics & Knowledge Conference*, pages 26–35. Association for Computing Machinery.
- Munzner, T. (2009). A Nested Model for Visualization Design and Validation. *IEEE Transactions on Visualization and Computer Graphics*, 15(6):921–928.
- Nielsen, J. (1993). *Usability Engineering*. Academic Press.
- Nielsen, J. (1994). Enhancing the explanatory power of usability heuristics. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 152–158. ACM Press.
- Parasuraman, R. and Riley, V. (1997). Humans and automation: Use, misuse, disuse, abuse. *Human Factors*, 39(2):230–253.
- Rollande, R. and Grundspenkis, J. (2013). Graph based framework and its implemented prototype for personalized study planning. In *2013 Second International Conference on E-Learning and E-Technologies in Education (ICEEE)*, pages 137–142. IEEE.
- Röpke, R., Judel, S., and Schroeder, U. (2024). Study path analyses for quality assurance and support of study planning: Approaches and advancements in the AIS-studyBuddy project. *Informatik Spektrum*, 47(3):97–104.
- Schulte, J., Fernandez De Mendonca, P., Martinez-Maldonado, R., and Buckingham Shum, S. (2017). Large scale predictive process mining and analytics of university degree course data. In *LAK '17: 7th International Learning Analytics and Knowledge Conference*, pages 538–539. Association for Computing Machinery.
- Schwendimann, B. A., Rodriguez-Triana, M. J., Vozniuk, A., Prieto, L. P., Boroujeni, M. S., Holzer, A., Gillet, D., and Dillenbourg, P. (2017). Perceiving Learning at a Glance: A Systematic Literature Review of Learning Dashboard Research. *IEEE Transactions on Learning Technologies*, 10(1):30–41.
- Teasley, S. D. (2017). Student Facing Dashboards: One Size Fits All? *Technology, Knowledge and Learning*, 22(3):377–384.
- Trippel, M. and Röpke, R. (2025). Developing a Graph-based Visualization of Elective Courses to Support Course Selection in Higher Education. In *23. Fachtagung Bildungstechnologien (DELFI)*.
- Verbert, K., Ochoa, X., De Croon, R., Dourado, R. A., and De Laet, T. (2020). Learning analytics dashboards: the past, the present and the future. In *Proceedings of the 10th International Conference on Learning Analytics and Knowledge (LAK '20)*, pages 35–40. Association for Computing Machinery.
- Vessey, I. (1991). Cognitive fit: A theory-based analysis of the graphs versus tables literature. *Decision Sciences*, 22(2):219–240.
- Zucker, R. (2009). ViCurriAS: a curriculum visualization tool for faculty, advisors, and students. *Journal of Computing Sciences in Colleges*, 25(2):138–145.